Green Computing (CSE407)

Project: Evaluating Transferability of LLM-learned Steering Vectors for Query Optimization from PostgreSQL to MySQL

Submitted by –

Group No	Name	ID	Contribution
08	**Rifa Azad**	**2022-1-60-265**	**45%**
	Md. Jubayer Ahmed	**2022-1-60-307**	**35%**
	Marjuk Ibna Belayet	**2022-1-60-383**	**10%**
	Jobayer Amin Rifat	**2022-1-60-084**	**10%**

Section: 02

Submitted to

Ahmed Abdal Shafi Rasel

Lecturer

Department of CSE

Date: 17 December 2025

Table of Contents

Abstract 2
1. Introduction 2
2. Literature Review 3
3. Problem Formulation and Hypothesis 7
3.1. Problem Statement 7
3.1.1. Gaps identified 8
3.2. Hypothesis 8
3.2.1. Hypothesis setup 9
4. Methodology 9
5. Experimental Setup 9
5.1. Systems 9
5.2. Query filtering rationale 10
5.3. Evaluation Metrics 10
6. Results 10
6.1. Summary of Results 10
6.2. Analysis 11
7. Limitations and Future Work 11
7.1. Limitations 11
7.2. Future Work 11
8. Conclusion 12
9. Repository & Screenshots 12
References 13

Abstract

Traditional database optimizers rely on statistics like cardinality (number of rows and columns) and indexes to choose the execution plan. They make decisions based on systems like features and heuristics. In this project we explore steering model based on PostgreSQL and implement it on MySQL database in a zero-shot way to evaluate whether MySQL perform like PostgreSQL or not. We train a simple binary classifier to predict our measurements based on the base line.
We measure the workload performance based on total latency and P90 tail latency. The difference in two different databases behavior may hint meaning between system can reduce consistency and limit gains, which we found to be non-effective from our experiments. Therefore, we can say that cross database query optimization is promising but due to lack of multi class steering, proper tuning, needs better alignment between DBMS action spaces and likely lightweight optimization technique that may useful both well-known Databases (PostgreSQL and MySQL) and reliable in practice.

1. Introduction

The growth of applications with large data, cloud platforms, large-scale storage systems along with the AI industry, has significantly increased the energy footprint of modern technology. As organizations rely on complex database ecosystems to support analytics, transactional workloads, the energy consumed by these systems has become a major concern in the pursuit of Green IT, especially for data centers (Guo et al., 2023). Governments globally are making sustainability a regulatory priority; energy efficiency now has become an important aspect to think about for technology also.

Most database operations are not energy-concerned due to large systems. Several database operations, such as query execution, indexing, and data access, are normally optimized for latency or throughput without considering power consumption. That leads to a big gap between performance optimization and energy efficiency. Conventional query optimizers use cost models that give good estimates on execution time but do not consider the amount of energy consumed during query processing. Learning-to-rank approaches have recently achieved very promising results for query plan selection; however, even these techniques still focus more on performance metrics rather than energy consumption.

The emergence of Large Language Models has opened new door in query optimization. Recent works show that LLMs can grasp query semantics, generate execution plans, and help with plan selection by reasoning. However, from an efficiency perspective, there are critical questions regarding the integration of LLMs into query optimization. LLM inference by itself is computationally expensive and energy-consuming. While LLMs may help in identifying better execution plans that reduce query runtime, there is a need to balance this against the energy cost of running the LLM optimizer. There is still a critical knowledge gap on whether LLM-based query optimization offers a net energy benefit when considering the LLM inference cost and energy savings due to improved query execution.

These works either present isolated measurements of database energy usage, performance-driven query optimization strategies, or exploratory studies on the capabilities of LLMs for optimization tasks. No comprehensive study has evaluated how efficiently LLM architectures can be designed for energy-aware query optimization in relational databases. Most LLM-based optimization frameworks are proof-of-concept implementations that do not address the fundamental trade-off between LLM inference energy and execution energy savings.

Building upon these observations, this work investigates how efficiently LLM architectures can be designed for query optimization in relational databases, focusing on the trade-off between the energy cost of LLM inference and the energy benefits of improved query execution plans.

2. Literature Review

As concern for energy awareness is increasing in the technology field, energy consumption in database systems has become a well explored area in research field. A systematic survey (Guo et al., 2023) categorized various database-level energy-saving techniques and highlighted the absence of integrated tools for developers. However, according to a Python-based benchmarking framework stated in (Milojković et al., 2025), the performance is the key indicator for database selection instead of energy, implying a gap in energy-aware benchmarking practices. Also, a survey (Lee et al., 2024) raised concerns for the trade-offs in energy-efficient methods, particularly in query optimization, which they noted as an area that has not been fully explored at the software level.

2.1. Early Observations and Energy Measurement Infrastructure

For standardized energy measurement infrastructure, (Michanan et al., 2017) introduced an energy-efficient framework for development process called Green C5, which helps to compare other benchmarks, query optimization, monitor CPU-heavy loops, and expensive

queries. The study (Miranskyy et al., 2018) analyzes 40 releases of MySQL and found that energy consumption and execution usually increase with maturity, thus challenging the assumption that software evolves towards efficiency. An instrumentation pipeline focused on precise energy measurement and profiling techniques was proposed by (Guo et al., 2024) that provided a strong empirical foundation for understanding how specific SQL operations contribute to energy consumption.

The study (Lella et al., 2024) shows the importance of practical observation in energy consumption based on variable performances across different database management systems depending on query structure, index usage, and the complexity of JOIN/UPDATE operations.

2.2. Energy-Aware Query Optimization Frameworks

2.2.1. Early Frameworks and Models

A general framework (Tu et al., n.d.), E^2DBMS, proposed to integrate energy-aware query optimization with a control-theoretic feedback loop for dynamic CPU tuning, initiating the research agenda for unified energy management in DBMSs. Later on, a regression-based model, EnerQuery, was proposed (Roukh et al., 2016) for predicting and optimizing the energy consumption of SQL queries that estimates the energy cost of individual operators and entire query plans.

The paper (Guo et al., 2017) introduced a green query optimization framework that integrates energy models into the query planner. Their results show that energy-efficient plans can reduce power usage without major performance loss.

2.2.2. Cost Prediction and Modeling

For realizing such energy-aware query optimization, (Saranya et al., 2018) targets a key enabler for energy-aware databases: accurate energy cost prediction for query optimization. The authors introduce a two-level modeling framework that incorporates a static energy model of database operators with a dynamic online model using Recursive Least Squares to adapt to changing conditions of the system. This paper contributes to the fundamental research in cost-based optimization by elevating energy to a first-class metric, with the intent to enhance the reliability of energy-efficient plan selection in dynamic environments.

A pipeline-level energy model has optimized the PostgreSQL query in (Roukh et al., 2019). In order to achieve energy-efficient query optimization, they follow parsing, rewrite, planning for optimization, and execute their results.

2.2.3. Learning-based Optimization Models

LLMs such GPT-4 (OpenAI et al., 2023), Deepseek (DeepSeek-AI et al., 2024), Gemini (Comanici et al., 2025) has been gaining a lot of attention in both general and technical fields. These models have demonstrated excellent capability in natural language along with solving complex problems which intrigued researchers to use LLMs for query optimization to solve the issue on energy efficiency in query optimization for DBMS.

This paper (Zhu et al., 2023) presents Lero, a learning-to-rank query optimizer that wraps around an existing cost-based optimizer without replacing it. Lero can be easily incorporated into PostgreSQL with only minimal modifications. In experiments on multiple benchmarks, it reduces execution time relative to the native optimizer by up to roughly 70%, outperforms several prior learned optimizers, and does so while continuously adapting to workload and data drift.

The paper (Akioyamen et al., 2024) examines LLMs' surprising utility for query optimization tasks, showing that LLM embeddings of queries contain semantic signals useful for plan selection and optimizer assistance. They embed query text using LLMs and use those embeddings as features for plan ranking/prediction or as inputs to simpler ML models. They compare against specialized learned-optimizer techniques and analyze when LLM features are most informative. In many cases, LLM-derived features can match or complement more complex learned optimizers, often with lower training overhead.

In (Sui et al., 2024) found that performance variation and developed a benchmark that leads to effective improvement in LLM performance to evaluate the table. (Tan et al., 2025) proposes an LLM-based query optimizer (LLM-QO) that uses LLM reasoning to assist or perform parts of query optimization within PostgreSQL's execution engine. The authors integrate LLM prompts/outputs into plan generation and selection, experiment with prompt design, and evaluate the feasibility and quality of plans produced or recommended by the LLM-assisted system. They analyze where LLMs succeed and where they fail. LLMs can produce plausible optimization suggestions and sometimes high-quality plans but remain brittle for some plan-space complexities. Prompt engineering and combination with cost estimates are critical. GPTuner was introduced (Giannakouris & Trummer, 2024) to read manuals as domain knowledge, Table-GPT (Li et al., 2024) showed ‘table-tuning’ to enhance LLM performance for table tasks.

LLMOpt (Yao et al., 2025) presents a two-part framework: generate plan candidates (LLMOpt(G)) and a list-wise selection model (LLMOpt(S)), aiming to use LLM reasoning that will help reducing search cost, and employs a selection mechanism that evaluates the list globally. The framework shows promise for reducing search overhead and improving plan quality in complex queries.

2.3. Trade-offs and Performance Considerations

A proactive energy-management model (HayamizuYuto et al., 2024) yielded up to 29.6% energy reduction at the cost of 7.6% slower operation, while integrating control over storage-device power state directly into the query execution, therefore showing an energy trade-off within query optimization. On the other hand, performance-based evaluation can lower energy use implicitly, as pointed out in (Stewart et al., 2025), which found that execution latency reduced up to 45-50% across traditional and learning-based query optimizers.

2.4. Optimization Techniques Across Different Systems

2.4.1. SQL & NoSQL Comparison

A work showed reducing I/O and network wait times sometimes may play a important role for energy efficiency by (Mahajan et al., 2019) showing that SQL and NoSQL engines exhibit different energy behaviors depending on workload characteristics based on speed and power and apply common query optimization in both systems.

2.4.2. Specific Strategies

A case study (Bellatreche et al., 2023) shows that even minor optimization decisions, such as join order or operator type, can alter the energy consumption. Similarly, (Cedeño et al., 2024) shows, with the help of the TPC-H benchmark, how different optimization techniques affect energy consumption in relational databases such as PostgreSQL. They work on Base, Index, Compression, and both (index + compression) optimization methods to evaluate their results. This work (Lindner et al., 2024) automates discovery, validation, and use of data dependencies beyond declared PK/FK such as functional dependencies, unique column combinations, and order dependencies for optimizing queries. The system discovers dependency candidates, validates applicability cheaply, and rewrites or optimizes plans accordingly. The paper implements a pipeline that extracts dependency candidates, validates them via lightweight checks, and integrates them into the optimizer to rewrite queries or choose better plans. Evaluated on TPC-DS, JOB and other benchmarks, automatically discovered dependencies yield large performance gains with reported geometric mean speedups around 29–35% on some benchmarks. The validation overhead is small relative to runtime benefits, and the system matches the performance of manual SQL rewrites without manual declarations.

(Alshareefi et al., 2025) includes Adaptive Query Optimization and Partition Aware Load Balancer (PALB) to reduce query delay and enhance energy efficiency. They use both indexing and compression under the (PALB+AQO) pipeline to evaluate the significant changes in terms of energy consumption. The paper (Liu et al., 2024) investigates optimization techniques for workloads that integrate relational data analytics with the serving of LLMs. It proposes several row/field reordering, deduplication, and batching strategies that maximize KV-cache reuse in LLM serving. The approach has been implemented within Apache Spark, using vLLM as a backend; it reorders rows and fields to enhance the cache hit rate of the LLMs and avoids redundant requests. The results indicate that significant reductions can be acheived in latency and cost for LLM inference by exploiting data arrangement and deduplication, showing query-level transformations may have a considerable impact on expensive inference stages.

2.4.3. Renewable Energy Integration and Carbon Awareness

Another perspective from (Chen et al., n.d.) discusses how to manage the query workload along with energy availability, especially with renewable sources. (Bachras & Jacobsen, 2025) introduces carbon-aware and eco-efficient database systems with a multi-dimensional optimization framework that accounts for carbon intensity of electricity, energy consumption, resource availability, and execution latency.

Paper	Datasets/benchmark	DBMS	Optimization technique

(Lella et al., 2024)	Netflix Userbase with 2500 rows, Online Retail with 541909 rows, and International Energy Statistics with 1189482 rows)	MySQL, PostgreSQL, MongoDB, and Couchbase	Open-source toolkit - tool called DBJoules
(Stewart et al., 2025)	TPC-H and TPC-DS	SQL, NoSQL, distributed cloud databases (MySQL, PostgreSQL, Oracle, MongoDB, Cassandra)	hybrid framework integrating machine learning-based optimization; Cost-Based Optimization (CBO) and modern AI-driven techniques
(HayamizuYuto et al., 2024)	TPC-H	PostgreSQL	Query execution with storage-state controls, energy aware cost modeling, builds on DVFS.
(Milojković et al., 2025)	Not specified clearly but kept focus on general query performance.	MySQL, MongoDB	Query performance benchmarking (Python-based, SDLC-independent approach with guidelines)
(Tu et al., n.d.)	TPC-H, TPC-C	PostgreSQL	Energy-aware query optimization, DVFS control, power-aware storage management.
(Miranskyy et al., 2018)	TPC-H	MySQL (InnoDB, MyISAM engines)	Analysis of code metrics (e.g., LOC) to predict energy use
(Saranya et al., 2018)	TPC-H, Astronomical dataset (SDSS)	PostgreSQL	Static and dynamic energy cost modeling for query plans.
(Guo et al., 2017)	TPC-H benchmark-style datasets (analytical workloads)	Custom experimental DBMS; relational query engine	Energy-aware cost model, energy-efficient operator selection, redesigned query optimizer
(Cedeño et al., 2024)	TPC-H	PostgreSQL 16	Indexing and data compression
(Roukh et al., 2019)	TPC-H	PostgreSQL	multi objective and cost based query optimization
(Alshareefi et al., 2025)	500 GB TIME series data	PostgreSQL	AQO AND PALB
(Roukh et al., 2016)	Synthetic datasets + execution logs for regression model training	PostgreSQL	Regression-based energy estimation for query pipelines; energy-aware plan selection; GUI-based energy hotspot analysis
(Zhu et al., 2023)	Multiple relational benchmarks (e.g., TPC-style and join-heavy workloads; exact set is in full paper)	Multiple relational benchmarks (e.g., TPC-style and join-heavy workloads; exact set is in full paper)	Multiple relational benchmarks (e.g., TPC-style and join-heavy workloads; exact set is in full paper)

3. Problem Formulation and Hypothesis

3.1. Problem Statement

Database Optimization is most obvious reason to optimize the database performance. As the data keeps growing day by day and also the application

become more interactive so, database optimization is an essential tool for handling tool without falling apart. Modern database query optimization is rely on handcraft tools models and tightly depends on specific database for this reason we can't optimize and generalize the database management systems. Recent work (Akioyamen et al., 2024) introduce a query optimization technique where it can produce different plan variant and hints to force the optimization towards plans and optimization characteristics. It use semantic information contained in raw SQL text and added embedded query vector to optimize the result. Instead of traditional optimization where it rely on statistics and make their decision based on features and heuristics, LLMSteer uses the semantic meaning of the Query for database optimization.

3.1.1. Gaps identified

Despite significant attention to energy-aware query optimization in relational databases and LLM-assisted query optimization, techniques the literature review reveals that as some paper managed to address one basic problem but fails to cover others, indicating a trade in this field.
Such as in (Akioyamen et al., 2024) the authors successfully managed to used embedded vectors to reduce latency in query optimization in PostgreSQL but they did not address how models will act in case data drift or workload changes, or how their proposed LLMSteer will behave in case of semantic changes.
Also, in some other works it is seen that they were able to tweak the LLM to optimize query execution but all of these work were always done in PostgreSQl environment, even the hint generated using LLM were based on PostgreSQL optimizer. MySQL is less explored DBMS system in Query optimization using LLM.

3.2. Hypothesis

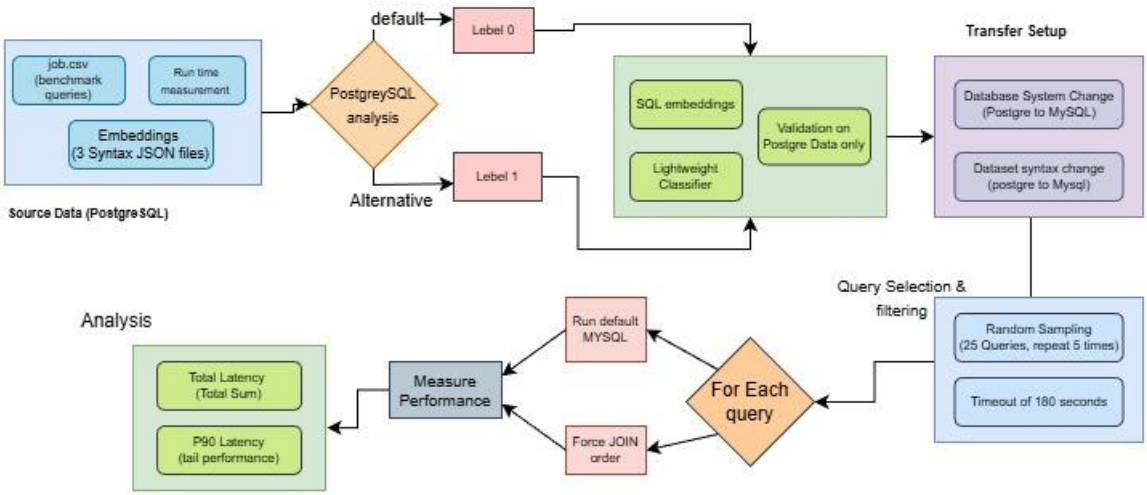
The hypothesis is that can steering vectors learned on PostgreSQL can do better in other DBMS like MySQL or DuckDB or not. The primary goal is to encode the query semantic from PostgreSQL and apply it to other DBMS to optimize the result. We have to use Zero-shot transfer to MySQL without using to any target system training data. This also mean not changing the model.
For fine-tuning steering mechanism there are some target data like run different size queries to monitor the performance on MySQL. The evaluation matrix that we followed to evaluate our result is total latency, P90 latency and based on the base line we have to judge if transfer works or not.

3.2.1. Hypothesis setup

- Our target was accomplished as DBMS is MySQL and apply steering vector on this system based on the steering vector used in PostgreSQL.
- Small fine tuning on the target DBMS(10,100,100 examples) to allow light engineering – which could not be followed due to hardware restrain.
- Use binary decision where its action goes default optimizer or force join order the hints used in LLMSteer was postgresql optimizer's.
- Evaluate total latency and P90 latency to monitor the optimization result

4. Methodology

This study aims to investigate efficient LLM architecture design for query optimization in relational databases, to reduce energy consumption.



5. Experimental Setup

5.1. Systems

- Dataset: Join Order Benchmark (JOB) (github)
- Labels are derived **from PostgreSQL executions (github)**, and never **never recomputed** for MySQL.
  - Each query is executed for the default optimizer and alternative join-

order (hints that suggest performance will increase.

- If any alternative yields a lower execution time than 2% (decided experimentally), the query is labeled:
  - 1 → alternative
  - 0 → default

- MySQL 8.0.44
- `/*+ JOIN_FIXED_ORDER() */` - was imposed for label 1

5.2. Query filtering rationale

Due to hardware constraints and long query runtimes, especially for join queries, we applied:

- join-count filtering
- pilot-based timeout filtering
- 25 queries with 5 time repetitions to reduce noise in query execution

5.3. Evaluation Metrics

Based on (Akioyamen et al., 2024), we also kept out metrics on:

- **Total latency:** sum of query execution times
- **P90 latency:** 90th percentile query latency *(90% of queries finish faster than given time)*

Comparing three strategies:

1. Default MySQL optimizer
2. Always-forced join order
3. Model-chosen (zero-shot steering)

6. Results

6.1. Summary of Results

Syntax Type of Embeddings	Total Latency (ms)	Improvement	Mode
A	+1.02M ms	43% slower)	forced much faster here
B	+1.03M ms	48% slower	forced faster
C	+1.09M ms	50% slower	forced faster

6.2. Analysis

Approximately 48% of queries (12 out of 25) timed out across all runs, hitting the execution limit. Therefore it is safe to say by this experiment that embedding trained on a DBMS can not be transfer to another DBMS and expect latency improvements, aka query more optimized but reduce the optimizations.

The zero-shot transfer in this scenario did not work.

7. Limitations and Future Work

7.1. Limitations

Even though the analysis was not positive but helped to understand whether embedded vectors transfer that was found in a DBMS will not perform similarly in another DBMS. But some restraints remain:

- The binary classification was not up to mark, as it gives the model very little option to check the queries on.
- As the large queries were not able to run completely, this creates a gap in query optimization decision.
- The hints were not the 48 hints used in LLMSteer from (Marcus et al., 2021). But forced one joint hint.
- Introduce cross-schema where train schema applicable for different table or relationships work better or not.

7.2. Future Work

By addressing these limitations, it is possible to see a positive result in the proposed system as (LiuHanwen et al., 2025) as they have seen a rise in improvement when they transfer their worked PostgreSQL work to MySQL environment.
Also, embeddings sometimes miss subtle changes in query, by addressing this while creating a common space that will be workable for both systems (PostgreSQL, MySQL) where the 48 hints will be useable in both DBMS will open the possibility to use the 48 hints removing the mess of binary labeling and forced one join operation.

8. Conclusion

The work that evaluates whether SQL embedding based steering decision train on PostgreSQL can work better and optimize and transfer Zero-shot to MySQL or not. We found that SQL embedding by vector wise semantic meaning may not be transferable across different DBMS. Zero-shot transfer is useful but limited in terms of our practical exhibition as we could not explore all hints, nor ran all queries completely due to hardware constraints. Database specific optimizer remains major challenge in terms of LLM.

9. Repository & Screenshots

Git link: https://github.com/rifa-zd/Steer-Vector-in-MySQL-407-project

Database changing to MySQL environment

```
mysql> DROP DATABASE job;
Query OK, 9 rows affected (0.34 sec)

mysql> CREATE DATABASE job;
Query OK, 1 row affected (0.01 sec)

mysql> USE job;
Database changed
mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| job                |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.01 sec)

mysql> SOURCE C:/jobdata/imdb/schema_mysql.sql;
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> USE job;
Database changed
mysql> LOAD DATA LOCAL INFILE 'C:/jobdata/imdb/title.csv'
    -> INTO TABLE title
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\n'
    -> IGNORE 1 ROWS;
Query OK, 2528311 rows affected, 65535 warnings (2 min 9.03 sec)
Records: 2528311  Deleted: 0  Skipped: 0  Warnings: 6435867

mysql>
mysql> SELECT COUNT(*) FROM title;
+----------+
| COUNT(*) |
+----------+
|  2528311 |
+----------+
1 row in set (1.56 sec)

mysql> LOAD DATA LOCAL INFILE 'C:/jobdata/imdb/name.csv'
    -> INTO TABLE name
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\n'
    -> IGNORE 1 ROWS;
Query OK, 4167490 rows affected, 65535 warnings (2 min 13.03 sec)
Records: 4167490  Deleted: 0  Skipped: 0  Warnings: 4167490

mysql>
mysql> SELECT COUNT(*) FROM name;
+----------+
| COUNT(*) |
+----------+
|  4167490 |
+----------+
1 row in set (2.67 sec)
```

```
+-----------------+------------+
| table_name      | real_rows  |
+-----------------+------------+
| cast_info       |  36244343  |
| movie_info      |  14835719  |
| movie_keyword   |   4523929  |
| name            |   4167490  |
| char_name       |   3140338  |
| person_info     |   2963663  |
| movie_companies |   2609128  |
| title           |   2528311  |
| movie_info_idx  |   1380034  |
| aka_name        |    901342  |
| aka_title       |    361471  |
| company_name    |    234996  |
| complete_cast   |    135085  |
| keyword         |    134169  |
| movie_link      |     29996  |
| info_type       |       112  |
| link_type       |        17  |
| role_type       |        11  |
| kind_type       |         6  |
| comp_cast_type  |         3  |
| company_type    |         3  |
+-----------------+------------+
21 rows in set (28.36 sec)
```

Results from VS code Terminal (Syntax A, Syntax B, Syntax C)

```
=== Workload Metrics (LLMSteer-style) ===
   mode      total_ms      p90_ms  total_impr_%  p90_impr_%
default 2,342,187.72 180,000.00          0.00        0.00
 forced 3,359,133.83 180,000.00        -43.42        0.00
 chosen 3,030,224.75 180,000.00        -29.38        0.00
```

```
Saved results to C:\Users\beaut\Desktop\LLMSteer\results\mysql_results_25_B.csv
(venv311) PS C:\Users\beaut\Desktop\LLMSteer> python analyze_results.py
=== Workload Metrics (LLMSteer-style) ===
   mode      total_ms      p90_ms  total_impr_%  p90_impr_%
default 2,170,562.61 180,000.00          0.00        0.00
 forced 3,205,282.26 180,000.00        -47.67        0.00
 chosen 2,810,894.66 180,000.00        -29.50        0.00

Saved: mysql_metrics_summary_B.csv, mysql_results_with_caps_B.csv
```

```
Saved results to C:\Users\beaut\Desktop\LLMSteer\results\mysql_results_25_C.csv
(venv311) PS C:\Users\beaut\Desktop\LLMSteer>
(venv311) PS C:\Users\beaut\Desktop\LLMSteer>
(venv311) PS C:\Users\beaut\Desktop\LLMSteer> python analyze_results.py
=== Workload Metrics (LLMSteer-style) ===
   mode      total_ms      p90_ms  total_impr_%  p90_impr_%
default 2,192,387.40 180,000.00          0.00        0.00
 forced 3,281,222.82 180,000.00        -49.66        0.00
 chosen 2,898,060.26 180,000.00        -32.19        0.00

Saved: mysql_metrics_summary_C.csv, mysql_results_with_caps_C.csv
```

References

Akioyamen, P., Yi, Z. & Marcus, R. (2024). *The Unreasonable Effectiveness of LLMs for Query Optimization*. https://arxiv.org/pdf/2411.02862

Alshareefi, H., Habib, B. H., Obaid, E. J., Ahmad, H. K. & Alsaidi, J. (2025). Database Optimization for Sustainable Energy Analytic. *International Journal on Informatics Visualization*, *9*(4), 1733–1742. https://doi.org/10.62527/JOIV.9.4.4381

Bachras, M. & Jacobsen, H. A. (2025). Environmental Footprints of Query Processing: A Vision for Sustainable Database Architectures. *Proceedings of the VLDB Endowment*, *18*(11), 4064–4072. https://doi.org/10.14778/3749646.3749676;CSUBTYPE:STRING:JOURNAL;SERIALTOPIC:TOPIC:ACM-PUBTYPE

Bellatreche, L., Djellali, F., MacYna, W. & Ordonez, C. (2023). Energy-Aware Query Processing: A Case Study on Join Reordering. *Proceedings - 2023 IEEE International Conference on Big Data, BigData 2023*, 3743–3752. https://doi.org/10.1109/BIGDATA59044.2023.10386332

Cedeño, E., Aguilera, A., Muñante, D., Correia, J., Guerrero, L., Sivira, C. & Cardinale, Y. (2024). An Evaluation of the Impact of End-to-End Query Optimization Strategies on Energy Consumption. *International Conference on Evaluation of Novel Approaches to Software Engineering, ENASE - Proceedings*, 657–665. https://doi.org/10.5220/0012708000003687

Chen, C., He, B., Tang, X., Chen, C. & Liu, Y. (n.d.). *Green Databases Through Integration of Renewable Energy*.

Comanici, G., Bieber, E., Schaekermann, M., Pasupat, I., Sachdeva, N., Dhillon, I., Blistein, M., Ram, O., Zhang, D., Rosen, E., Marris, L., Petulla, S., Gaffney, C., Aharoni, A., Lintz, N., Pais, T. C., Jacobsson, H., Szpektor, I., Jiang, N.-J., … Helmholz, W. (2025). *Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities*. https://arxiv.org/pdf/2507.06261

DeepSeek-AI, Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., … Pan, Z. (2024). *DeepSeek-V3 Technical Report*. https://arxiv.org/pdf/2412.19437

Giannakouris, V. & Trummer, I. (2024). Demonstrating λ-Tune: Exploiting Large Language Models for Workload-Adaptive Database System Tuning. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 508–511. https://doi.org/10.1145/3626246.3654751;TOPIC:TOPIC:CONFERENCE-COLLECTIONS

Guo, B., Wu, J., Pu, Y., Zhang, J. & Yu, J. (2024). Energy consumption estimation and profiling for queries in distributed database systems based on a bottom-up comprehensive energy model. *Future Generation Computer Systems*, *159*, 379–394. https://doi.org/10.1016/J.FUTURE.2024.04.059

Guo, B., Yu, J., Liao, B., Yang, D. & Lu, L. (2017). A green framework for DBMS based on energy-aware query optimization and energy-efficient query processing. *Journal of Network and Computer Applications*, *84*, 118–130. https://doi.org/10.1016/J.JNCA.2017.02.015

Guo, B., Yu, J., Yang, D., Leng, H. & Liao, B. (2023). Energy-Efficient Database Systems: A Systematic Survey. *ACM Computing Surveys*, *55*(6). https://doi.org/10.1145/3538225

HayamizuYuto, KitsuregawaMasaru & GodaKazuo. (2024). Proactive Energy Management in Database Systems. *ACM SIGENERGY Energy Informatics Review*, *4*(5), 154–159. https://doi.org/10.1145/3727200.3727223

Lee, S. U., Fernando, N., Lee, K. & Schneider, J. G. (2024). A survey of energy concerns for software engineering. *Journal of Systems and Software*, *210*, 111944. https://doi.org/10.1016/J.JSS.2023.111944

Lella, H. S., Chattaraj, R., Chimalakonda, S. & Kurra, M. (2024). Towards Comprehending Energy Consumption of Database Management Systems - A Tool and Empirical Study. *ACM International Conference Proceeding Series*, 272–281. https://doi.org/10.1145/3661167.3661174;PAGE:STRING:ARTICLE/CHAPTER

Li, P., He, Y., Yashar, D., Cui, W., Ge, S., Zhang, H., Rifinski Fainman, D., Zhang, D. & Chaudhuri, S. (2024). Table-GPT: Table Fine-tuned GPT for Diverse Table Tasks. *Proceedings of the ACM on Management of Data*, *2*(3), 1–28. https://doi.org/10.1145/3654979;TAXONOMY:TAXONOMY:ACM-PUBTYPE;PAGEGROUP:STRING:PUBLICATION

Lindner, D., Ritter, D. & Naumann, F. (2024). *Enabling Data Dependency-based Query Optimization*. *1*. https://arxiv.org/pdf/2406.06886

Liu, S., Biswal, A., Kamsetty, A., Cheng, A., Schroeder, L. G., Patel, L., Cao, S., Mo, X., Stoica, I., Gonzalez, J. E. & Zaharia, M. (2024). *Optimizing LLM Queries in Relational Data Analytics Workloads*. https://arxiv.org/pdf/2403.05821

LiuHanwen, ZhangQihan, MarcusRyan & SabekIbrahim. (2025). SEFRQO: A Self-Evolving Fine-Tuned RAG-Based Query Optimizer. *Proceedings of the ACM on Management of Data*, *3*(6), 1–27. https://doi.org/10.1145/3769826

Mahajan, D., Blakeney, C. & Zong, Z. (2019). Improving the energy efficiency of relational and NoSQL databases via query optimizations. *Sustainable Computing: Informatics and Systems*, *22*, 120–133. https://doi.org/10.1016/J.SUSCOM.2019.01.017

Marcus, R., Negi, P., Mao, H., Tatbul, N., Alizadeh, M. & Kraska, T. (2021). Bao: Making Learned Query Optimization Practical. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1275–1288. https://doi.org/10.1145/3448016.3452838;PAGE:STRING:ARTICLE/CHAPTER

Michanan, J., Dewri, R. & Rutherford, M. J. (2017). GreenC5: An adaptive, energy-aware collection for green software development. *Sustainable Computing: Informatics and Systems*, *13*, 42–60. https://doi.org/10.1016/J.SUSCOM.2016.11.004

Milojković, K., Spalević, P., Vasić, N., Milojković, N. & Milojković, H. (2025). SDLC-Independent Python-Based Query Performance Benchmarking Approach and Practical Optimal Database Selection Guidelines. *Sinteza 2025 - International Scientific Conference on Information Technology, Computer Science, and Data Science*, 536–543. https://doi.org/10.15308/SINTEZA-2025-536-543

Miranskyy, A. V., Al-zanbouri, Z., Godwin, D. & Bener, A. B. (2018). Database engines: Evolution of greenness. *Journal of Software: Evolution and Process*, *30*(4). https://doi.org/10.1002/SMR.1915

OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., Avila, R., Babuschkin, I., Balaji, S., Balcom, V., Baltescu, P., Bao, H., Bavarian, M., Belgum, J., … Zoph, B. (2023). *GPT-4 Technical Report*. https://arxiv.org/pdf/2303.08774

Roukh, A., Bellatreche, L. & Ordonez, C. (2016). EnerQuery: Energy-aware query processing. *International Conference on Information and Knowledge Management, Proceedings*, *24-28-October-2016*, 2465–2468. https://doi.org/10.1145/2983323.2983334;WGROUP:STRING:ACM

Roukh, A., Bellatreche, L. & Tziritas, N. (2019). OBRE: OFFER-BORROW-REFORM-EVALUATE INITIATIVES FOR GREEN DBMSS. *Azerbaijan Journal of High Performance Computing*, *2*(1), 39–63. https://doi.org/10.32010/26166127.2019.2.1.39.63

Saranya, M. P., Soni Priyanka, K., Poonam, S. & Tamil Thendral, M. (2018). Energy Efficient Query Processing in Green Database. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology © 2018 IJSRCSEIT*, *3*(3), 1095–1100.

Stewart, N., Adams, D. & Foster, U. (2025). Empirical Analysis of Query Optimization Strategies for Reducing Execution Latency in Large-Scale Databases. *Journal of Innovation in Governance and Business Practices*, *1*(1), 88–117. https://jigbp.com/index.php/jigbp/article/view/4

Sui, Y., Zhou, M., Zhou, M., Han, S. & Zhang, D. (2024). Table Meets LLM: Can Large Language Models Understand Structured Table Data? A Benchmark and Empirical Study. *WSDM 2024 - Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, 645–

654. https://doi.org/10.1145/3616855.3635752

Tan, J., Zhao, K., Li, R., Yu, J. X., Piao, C., Cheng, H., Meng, H., Zhao, D. & Rong, Y. (2025). Can Large Language Models Be Query Optimizer for Relational Databases? *Proceedings of the VLDB Endowment*, *17*(10). https://arxiv.org/pdf/2502.05562

Tu, Y.-C., Wang, X. & Zeng, B. (n.d.). *A System for Energy-Efficient Data Management*.

Yao, Z., Li, H., Zhang, J., Li, C. & Chen, H. (2025). *A Query Optimization Method Utilizing Large Language Models*. *14*(1). https://arxiv.org/pdf/2503.06902

Zhu, R., Pfadler, A., Chen, W., Wu, Z., Ding, B., Zhou Jingren.Zhou@Alibaba-Inc.Com, J. & Chen, X. (2023). Lero: A Learning-to-Rank Query Optimizer. *Proceedings of the VLDB Endowment*, *16*(6), 1466–1479. https://doi.org/10.14778/3583140.3583160;CSUBTYPE:STRING:JOURNAL;ISSUE:ISSUE:DOI